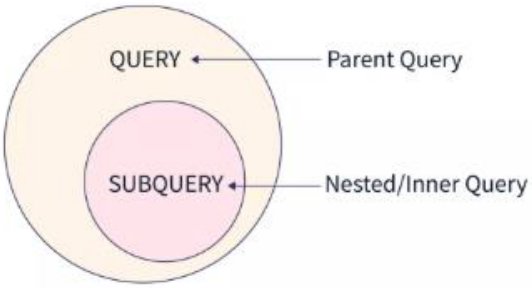
ADVANCE SQL QUERY IMPLEMENTATION

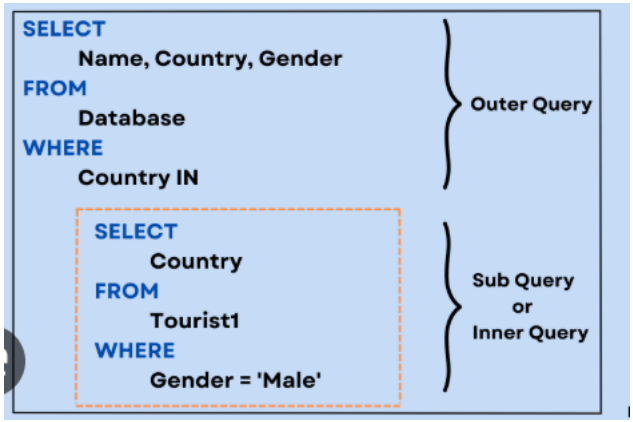
Pepperfry Furniture E-Commerce Database Management System

1. Subqueries and Nested Queries

A **Subquery (Nested Query)** is a query written inside another SQL query. It is used to retrieve data from one table and use that result in the main query.



Basic Structure



Types of Subqueries

Subquery	Description
Single-Row Subquery	Returns one row of results that consists of one column to the outer query
Multiple-Row Subquery	Returns more than one row of results to the outer query
Multiple-Column Subquery	Returns more than one column of results to the outer query
Correlated Subquery	References a column in the outer query. Executes the subquery once for every row in the outer query

Example

Find the products whose price is greater than the average product price

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price > (SELECT AVG(Price) FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------
     PRICE
----------
         1
Sofa
     27000

         2
Dining Table
     18000

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------
     PRICE
----------

         4
Bed
     30000

         7
Wardrobe
```

Explanation:

- Inner Query: SELECT AVG(Price) FROM Product calculates the average price.
- Outer Query: Displays products whose price is greater than the average price.

2 . Find Products Above Average Price

This query is used to identify products whose prices are greater than the average price of all products using the AVG() function.

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price > (SELECT AVG(Price) FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------
     PRICE
----------
         1
Sofa
     27000

         2
Dining Table
     18000

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------
     PRICE
----------

         4
Bed
     30000

         7
Wardrobe
```

Explanation:

- Inner Query: Calculates the average price of all products using AVG().
- Outer Query: Displays products whose price is greater than the average price.

2.1 Find Products Below Average Price

This query is used to identify products whose prices are less than the average price of all products using the AVG() function.

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price < (SELECT AVG(Price) FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE
----------
         3
Office Chair
      7500

         5
Bookshelf
      8500

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE
----------

         6
Coffee Table
      6500

         8
TV Unit
```

3 . Identify Customers with Maximum Purchases

This query is used to identify the customer who has made the highest purchase using the MAX() function and subqueries.

```
SQL> SELECT Customer_ID, Customer_Name
  2  FROM Customer
  3  WHERE Customer_ID IN (
  4      SELECT Customer_ID
  5      FROM Orders
  6      WHERE Total_Amount = (
  7          SELECT MAX(Total_Amount)
  8          FROM Orders
  9      )
 10  );

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------
          5 Karthik
```

Find the Maximum Purchase

This query is used to find the highest purchase amount from the Orders table using the MAX() aggregate function.

```
SQL> SELECT Order_ID, Customer_ID, Total_Amount
  2  FROM Orders
  3  WHERE Total_Amount = (
  4      SELECT MAX(Total_Amount)
  5      FROM Orders
  6  );

  ORDER_ID CUSTOMER_ID TOTAL_AMOUNT
---------- ----------- ------------
         5           5        45000
```

3.1 Display Customer Details with Maximum Purchase

This query is used to retrieve the complete customer details of the customer who made the maximum purchase.

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.Customer_Name,
  4      SUM(O.Total_Amount) AS Total_Purchase
  5  FROM Customer C
  6  JOIN Orders O
  7  ON C.Customer_ID = O.Customer_ID
  8  GROUP BY
  9      C.Customer_ID,
 10      C.Customer_Name
 11  HAVING SUM(O.Total_Amount) = (
 12      SELECT MAX(Total_Purchase)
 13      FROM (
 14          SELECT SUM(Total_Amount) AS Total_Purchase
 15          FROM Orders
 16          GROUP BY Customer_ID
 17      )
 18  );

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_PURCHASE
----------- -------------------------------------------------- --------------
          5 Karthik                                                     45000
```

4 . Complex Business Queries

Complex Business Queries are SQL queries used to retrieve and analyze data from multiple tables using different SQL operations such as JOIN, GROUP BY, HAVING, subqueries, and aggregate functions to solve real-world business problems.

4.1 Customer Who Placed Orders Above Average Order Value

This query is used to identify customers who placed orders with an amount greater than the average order value using the AVG() function.

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.Customer_Name,
  4      O.Order_ID,
  5      O.Total_Amount
  6  FROM Customer C
  7  JOIN Orders O
  8  ON C.Customer_ID = O.Customer_ID
  9  WHERE O.Total_Amount > (
 10      SELECT AVG(Total_Amount)
 11      FROM Orders
 12  );

CUSTOMER_ID CUSTOMER_NAME                                      ORDER_ID
----------- -------------------------------------------------- ----------
TOTAL_AMOUNT
------------
          3 Rahul                                                       3
       32000

          5 Karthik                                                     5
       45000

          7 Vijay                                                       7
       28000


CUSTOMER_ID CUSTOMER_NAME                                      ORDER_ID
----------- -------------------------------------------------- ----------
TOTAL_AMOUNT
------------
          9 Ajay                                                        9
       37500
```

5 . Find Products Whose Sales are Above Average

This query is used to identify products whose total sales are greater than the average sales of all products

```
SQL> SELECT
  2      P.Product_ID,
  3      P.Product_Name,
  4      SUM(OD.Quantity * P.Price) AS Total_Sales
  5  FROM Product P
  6  JOIN Order_Details OD
  7  ON P.Product_ID = OD.Product_ID
  8  GROUP BY
  9      P.Product_ID,
 10      P.Product_Name
 11  HAVING SUM(OD.Quantity * P.Price) > (
 12      SELECT AVG(Total_Sales)
 13      FROM (
 14          SELECT SUM(OD.Quantity * P.Price) AS Total_Sales
 15          FROM Order_Details OD
 16          JOIN Product P
 17          ON OD.Product_ID = P.Product_ID
 18          GROUP BY OD.Product_ID
 19      )
 20  );

PRODUCT_ID
----------
PRODUCT_NAME
------------------------------------------------------------
TOTAL_SALES
-----------
          1
Sofa
     108000

          7
Wardrobe
```

6 . Customer Purchase Report

A Customer Purchase Report is used to display each customer's total number of orders and total purchase amount using aggregate functions such as COUNT() and SUM().

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.Customer_Name,
  4      COUNT(O.Order_ID) AS Total_Orders,
  5      SUM(O.Total_Amount) AS Total_Purchase
  6  FROM Customer C
  7  JOIN Orders O
  8  ON C.Customer_ID = O.Customer_ID
  9  GROUP BY
 10      C.Customer_ID,
 11      C.Customer_Name
 12  ORDER BY
 13      Total_Purchase DESC;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
TOTAL_PURCHASE
--------------
          5 Karthik                                                       1
         45000

          9 Ajay                                                          1
         37500

          3 Rahul                                                         1
         32000


CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
TOTAL_PURCHASE
--------------
          7 Vijay                                                         1
```

7 . Advanced Product Sales Report

An Advanced Product Sales Report is used to analyze product performance by displaying total quantity sold, total sales, and average product price.

```
SQL> SELECT
  2      P.Product_ID,
  3      P.Product_Name,
  4      SUM(OD.Quantity) AS Total_Quantity_Sold,
  5      SUM(OD.Quantity * P.Price) AS Total_Sales,
  6      AVG(P.Price) AS Average_Price
  7  FROM Product P
  8  JOIN Order_Details OD
  9  ON P.Product_ID = OD.Product_ID
 10  GROUP BY
 11      P.Product_ID,
 12      P.Product_Name
 13  ORDER BY
 14      Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
------------------- ----------- -------------
          1
Sofa
                    4      108000         27000

          7
Wardrobe
                    2       44000         22000
```

8 . Category-wise Advanced Sales Report

A Category-wise Advanced Sales Report is used to analyze sales performance for each product category by calculating total products, total quantity sold, total sales, and average price.

```
SQL> SELECT
  2      C.Category_ID,
  3      C.Category_Name,
  4      COUNT(DISTINCT P.Product_ID) AS Total_Products,
  5      SUM(OD.Quantity) AS Total_Quantity_Sold,
  6      SUM(OD.Quantity * P.Price) AS Total_Sales,
  7      AVG(P.Price) AS Average_Price
  8  FROM Category C
  9  JOIN Product P
 10  ON C.Category_ID = P.Category_ID
 11  JOIN Order_Details OD
 12  ON P.Product_ID = OD.Product_ID
 13  GROUP BY
 14      C.Category_ID,
 15      C.Category_Name
 16  ORDER BY
 17      Total_Sales DESC;

CATEGORY_ID CATEGORY_NAME                                      TOTAL_PRODUCTS
----------- -------------------------------------------------- --------------
TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
------------------- ----------- -------------
          1 Sofa                                                            1
                  4      108000         27000

          7 Dining Table                                                    1
                  2       44000         22000

          3 Chair                                                           2
                  3       31000         11750
```